

Intermediate Report

Prepared by: Alexandra Leung Dat Wan (2410909), Bhoovana Dinoo (2410920), Kentish Thummanah (2411831), Luvish Kumar Lokye (2410762), Nahla Dinmahamed (2414317). All pages were prepared by one team member, name and student ID were mentioned, to show the distribution of the report.

This intermediate report presents the progress we have made so far in our Fake News Detection model. We described the setup of our dataset and pre-processing methods. Furthermore, we discuss the challenges encountered, possible improvements to our initial approach, and outline the experiments we plan to conduct.

1. Cleaning phase (Part 1) – Luvish Kumar Lokye (2410762)

What progress have you made towards your proposed goal?

Firstly, we created a code to analyze our dataset. Our code looks for data that needs to be cleaned as shown in fig1.1. When executing the code, we can see there are 20000 rows which need to be cleaned.

In the cleaning algorithm, we used pandas library from Python to import the dataset and re library to modify the dataset. We also made use of the tqdm library to display a progress bar.



Fig 1.1

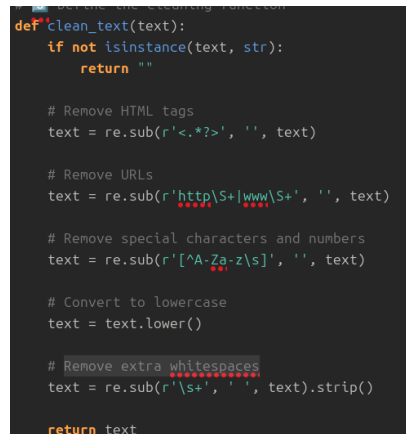


Fig 1.2

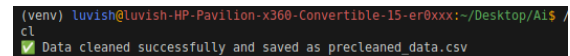


Fig 1.3

If you tried some basic approaches: what worked well and what did not?

We made use of an algorithm called cleanfinalscript.py shown in fig 1.2 which:

1. Removes HTML tags
2. Removes URLs
3. Convert the texts in the dataset to lowercase
4. Removes extra whitespaces

This approach worked best for us and helped us clean our dataset effectively for the next phase.

What could be done to improve the basic approaches?

After conducting further research, we deduced that we can further enhance the data cleaning by expanding the code to lemmatize the text and removing stop words, this is done in the cleaning phase 2.

What experiments have you run and are you planning to run to demonstrate effectiveness?

The experiment we've done so far is that we analyzed the raw data then we cleaned the data and saved it as precleaned_data.csv. On the next page, we will discuss how we enhance the dataset cleaning and there will be an experiment which will demonstrate the effectiveness of our cleaning phase.

2. Cleaning phase (Part 2) – Kentish Thummanah (2411831)

What progress have you made towards your proposed goal?

Finally, for the data cleaning phase, after following the process outlined by the previous team member, this code identifies data that requires cleaning based on the algorithm implemented in the script “cleanfinalscript.py”, as illustrated in Figure 2.1. We carried out the following steps:

```
# Load the spaCy model
nlp = spacy.load("en_core_web_sm")

def clean_text(text):
    # Process the text with spaCy
    doc = nlp(text)

    # Lemmatize, remove stop words
    cleaned_tokens = [token.lemma_ for token in doc if not token.is_stop and token.is_alpha]

    # Join tokens back into a single string from ['fake', 'text'] to 'fake text'
    cleaned_text = ' '.join(cleaned_tokens)

    return cleaned_text
```

Fig 2.1

- Loading the “precleaned.csv” file using pandas library
- Process the text using spaCy library
- Lemmatize the text, which is, bringing words to base form but still as text, for example, “running faster” will be converted to “run faster”
- Remove stop words such as “the”, “is”, “and”
- Saving the output in a new csv file called “cleaned_dataset.csv”



Fig 2.2

The progress we have made so far is that we have cleaned fully the data based on algorithms suitable for text datatype dataset. Fig 2.2 shows the progress bar, using tqdm library, and that each column had been cleaned for the next stage.

If you tried some basic approaches, what worked well and what did not?

The data has not been cleaned fully as there are still 18 dirty rows in the column title in the dataset as shown below in fig 2.3.

3.What could be done to improve the basic approaches?

After some research, we concluded that achieving 0 dirty row can sometimes remove meaningful content, thus achieving a 99.91% (from 20 000 dirty rows to 18 dirty rows) effectiveness in cleaning data is acceptable.

4. What experiments have you run and are you planning to run to demonstrate the effectiveness?

The experiment we’ve done to see if our data was cleaned is shown in fig 2.2 and fig 2.3. We made use of an algorithm called analyse.py which helped us to visualise the comparison and effectiveness achieved between the raw dataset and the cleaned dataset.



Fig 2.3

3. Tokenization phase – Nahla Dinmahamed (2414317)

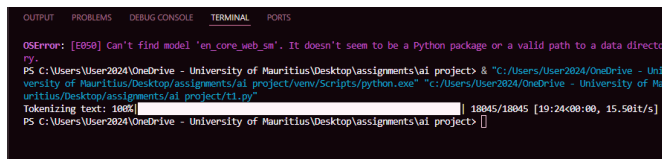
The spacy library, which is often used in natural language processing, is imported and its English model is used. The model of medium size is chosen to opt for a balance between fast performance and accuracy. Along with it, the pandas library is used to handle and process the dataset efficiently, as mentioned above.

```
1 import pandas as pd
2 import spacy
3 from tqdm import tqdm
4
5 # Load SpaCy model
6 nlp = spacy.load("en_core_web_md")
7
8 # Read the CSV file
9 df = pd.read_csv("cleaned_dataset.csv")
10
11 # Register tqdm with pandas apply
12 tqdm.pandas(desc="Tokenizing text")
13
14 # Define tokenization function
15 def tokenize_text(text):
16     doc = nlp(str(text)) # convert to string in case there are NaNs
17     return [token.text for token in doc if not token.is_punct and not token.is_space]
18
19 # Apply tokenization with a progress bar
20 df["tokens"] = df["text"].progress_apply(tokenize_text)
21
22 # Save the tokenized dataset
23 df.to_csv("tokenized_dataset.csv", index=False)
24
```

fig 3.1: code for tokenization

What progress have you made towards your proposed goal?

The CSV file named *cleaned_dataset.csv* was loaded using pandas, and the SpaCy model *en_core_web_md* was applied to perform tokenization on each article. The dataset has been successfully tokenized and saved as *tokenized_dataset.csv*, which will be used in the next step toward building the fake news detection model.



```
OUTPUT  PROBLEMS  DEBUG CONSOLE  TERMINAL  PORTS
OSError: [E050] Can't find model 'en_core_web_sm'. It doesn't seem to be a Python package or a valid path to a data directory.
PS C:\Users\User2024\OneDrive - University of Mauritius\Desktop\assignments\ai project> & "C:\Users\User2024\OneDrive - University of Mauritius\Desktop\assignments\ai project\venv\Scripts\python.exe" "C:\Users\User2024\OneDrive - University of Mauritius\Desktop\assignments\ai project\t1.py"
Tokenizing text: 100% | 18045/18045 [19:24:00:00, 15.50it/s]
PS C:\Users\User2024\OneDrive - University of Mauritius\Desktop\assignments\ai project>
```

fig 3.2: tokenizing the text and saving the file as *tokenized_dataset.csv*

2. If you tried some basic approaches: what worked well and what did not?

The SpaCy tokenization process worked effectively in extracting meaningful words from each article by removing punctuation marks and spaces. However, handling missing values and rare words required additional steps to ensure that only relevant tokens were included.

3. What could be done to improve the basic approaches?

The tokenization process can be improved by using more advanced models or embeddings could also enhance the representation of word relationships and context.

4. What experiments have you run and are you planning to run to demonstrate the effectiveness?

The tokenized dataset will be used to train and evaluate various machine learning models for fake news detection. Planned experiments will compare different model performances to determine which provides the highest accuracy and reliability.

4. Deduplication of the dataset – Bhoovana Dinoo (2410920)

Progress made towards the goal:

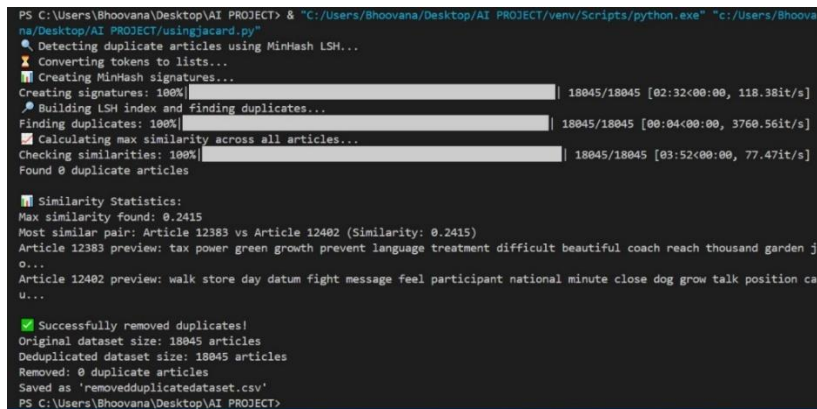
In this phase, we focused on identifying and removing similar or duplicate articles from the dataset. After the tokenization step was completed, we worked on applying a similarity measure that compares the tokenized articles directly. To achieve this, we implemented the Jaccard distance method, which evaluates the similarity between two sets of tokens based on their intersection and union. This approach allowed us to detect repeated or overlapping articles more effectively and hence we were able to clean the dataset for the final stage of our project.

Basic approaches tried; what worked and what didn't?

Initially, we experimented with the cosine similarity approach to measure similarity between article vectors. While cosine similarity is commonly used in vector-based representations, it was returning to a max similarity level of 0, hence it wasn't accurate. We decided to shift to Jaccard distance and use the tokenized data instead of the vectorized data, the results improved significantly for token-level comparisons and returned a value of 0.2415 for the max similarity. However, we noticed that the code gave an estimated running time of 7 hours.

Possible improvements to the basic approach

To fix the large running time issue, we implemented batch processing to cut the time significantly. Additionally, we used a hash-based deduplication using MinHash, it is a technique that creates a compact signature for a set of tokens, which is created using batch processing. This helped us improve our basic approach, Jaccard similarity. Figure 4.1 shows after the execution of the deduplication algorithm, also shows the maximum similarity. It is then saved to a file, as output "removedduplicateddataset.csv" which will be used in the further step to be converted to vectors.



```
PS C:\Users\Bhoovana\Desktop\AI PROJECT> & "c:/Users/Bhoovana/Desktop/AI PROJECT/venv/Scripts/python.exe" "c:/Users/Bhoovana/Desktop/AI PROJECT/usingjaccard.py"
Detecting duplicate articles using MinHash LSH...
Converting tokens to lists...
Creating MinHash signatures...
Creating signatures: 100% | 18045/18045 [02:32<00:00, 118.38it/s]
Building LSH index and finding duplicates...
Finding duplicates: 100% | 18045/18045 [00:04<00:00, 3760.56it/s]
Calculating max similarity across all articles...
Checking similarities: 100% | 18045/18045 [03:52<00:00, 77.47it/s]
Found 0 duplicate articles

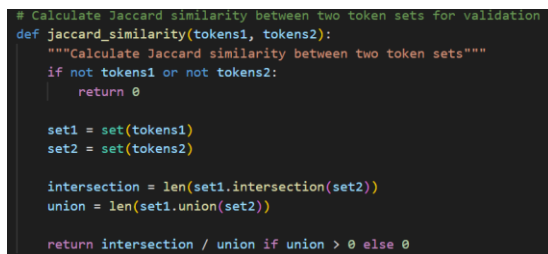
Similarity Statistics:
Max similarity found: 0.2415
Most similar pair: Article 12383 vs Article 12402 (Similarity: 0.2415)
Article 12383 preview: tax power green growth prevent language treatment difficult beautiful coach reach thousand garden j
o...
Article 12402 preview: walk store day datum fight message feel participant national minute close dog grow talk position ca
u...

Successfully removed duplicates!
Original dataset size: 18045 articles
Deduplicated dataset size: 18045 articles
Removed: 0 duplicate articles
Saved as 'removedduplicateddataset.csv'
PS C:\Users\Bhoovana\Desktop\AI PROJECT>
```

Figure 4.1

Experiments conducted and future plans

We experimented with both cosine similarity and Jaccard distance and compared their effectiveness. We finalized using **Jaccard distance** as it was more efficient. After this stage, tokens from the deduplicated dataset will be converted to vectors, and cosine similarity will still be used to calculate the weight of the article and hence, help determine whether the article is fake or not.



```
# Calculate Jaccard similarity between two token sets for validation
def jaccard_similarity(tokens1, tokens2):
    """Calculate Jaccard similarity between two token sets"""
    if not tokens1 or not tokens2:
        return 0

    set1 = set(tokens1)
    set2 = set(tokens2)

    intersection = len(set1.intersection(set2))
    union = len(set1.union(set2))

    return intersection / union if union > 0 else 0
```

Figure 4.2 shows part of the code implemented in Python, that calculates the Jaccard distance and hence uses intersection and union.

5. Converting the tokens into vectors – Alexandra Leung Dat Wan(2410909)

Progress made towards the goal:

During this phase, the tokens are transformed to a mathematical representation. The 'tokens' column, from removeduplicatedataset.csv is processed. spacy's model is used to map each word to its 300-dimensional vector which conveys statistical patterns and relationships between words, for e.g. 'tax' and 'government'. An average vector is then calculated to represent the whole article. This process is repeated for each article and the results saved to 'vectorized_data.csv'.

```
def tokens_to_vector(tokens_list):
    # Checking for empty or invalid data
    if not tokens_list or pd.isna(tokens_list):
        return ','.join(['0'] * 300)
    # Get vectors for all tokens that have vectors
    vectors = []

    # loops through each token in a row(tokens_list)
    for token in tokens_list:
        # assigns each word a vector by looking up in the spaCy dictionary
        word = nlp.vocab[str(token).lower()]
        # Some rare words might not have vectors
        if word.has_vector:
            vectors.append(word.vector)

    # Return average vector or zeros if no vectors found
    if vectors:
        avg_vector = np.mean(vectors, axis=0)
        # Convert directly to comma-separated string
        return ','.join(map(str, avg_vector))
    else:
        return ','.join(['0'] * 300)

# Read the tokenized CSV
df = pd.read_csv("tokenized_dataset.csv")

# Convert tokens to vectors with progress bar
print("🔄 Converting tokens to vectors...")
tqdm.pandas(desc="Processing")
df['vectors'] = df['tokens'].progress_apply(tokens_to_vector)
```

Figure 5.1

Basic approaches tried; what worked and what didn't?

We have experimented with TF-IDF which gives weight to frequent and significant words but realised that it cannot demonstrate related words like 'election' and 'vote'. We favoured spacy since vectors can represent the semantics of words.

What could be done to improve the basic approaches?

The current model cannot differentiate between a word with different meanings (like 'bank' which can either mean a financial institution or the edge of a river). By using a more contextual model, a vector can be calculated through the context of a sentence and better represent a topic of an article.

What experiments have you run and are you planning to run to demonstrate effectiveness?

We have successfully transformed our dataset of articles to its numerical representation and plan to use the file 'vectorized_data.csv' to train our system. An unknown article can be labelled as real or fake by comparing it to the training set using cosine similarity and obtaining a suitable prediction. By testing the system with new articles, we can determine its performance by checking for accuracy and precision.